TECHNICAL DOCUMENTATION

Project Engine

Comprehensive technical documentation for a full-stack hiring and project collaboration platform, covering architecture, database design, API reference, and deployment.

VERSION

1.0 • April 2026

STACK

PHP • SQL Server • HTML/CSS/JS

Table of Contents

1. Executive Summary 3

2. Technology Stack 3

3. Architecture and System Design 4

3.1 Three-Tier Architecture 4

3.2 Authentication and Authorization 5

3.3 Data Flow Patterns 5

4. Database Design 6

4.1 Schema Overview 6

4.2 Entity Relationship Summary 6

4.3 Table Definitions 7

4.3.1 Users 7

4.3.2 Developers 7

4.3.3 Clients 8

4.3.4 Projects 8

4.3.5 Supporting Tables 9

4.4 Indexes 10

5. API Reference 10

5.1 Authentication Endpoints 10

5.2 Project Management Endpoints 11

5.3 Developer and Hiring Endpoints 11

5.4 Workspace Endpoints 12

5.5 Messaging Endpoints 12

5.6 Profile and Account Endpoints 13

6. Frontend Documentation 14

6.1 Page Inventory 14

6.2 CSS Design System 14

6.3 JavaScript Architecture 15

7. User Flows 15

7.1 Client Journey 15

7.2 Developer Journey 16

7.3 Dual Hiring Pathways 16

8. Security Considerations 17

8.1 Implemented Security Measures 17

8.2 Identified Security Gaps 17

9. Known Issues and Limitations 18

9.1 Functional Issues 18

9.2 Architectural Limitations 18

10. Setup and Installation Guide 19

10.1 Prerequisites 19

10.2 Installation Steps 19

10.3 Configuration 19

11. File Dependency Map 20

1. Executive Summary

ProjectEngine is a full-stack web application designed to facilitate the connection between clients who need software projects built and developers who possess the technical skills to build them. The platform serves as a dedicated technical hiring and project collaboration ecosystem, providing role-based authentication, developer discovery with skill-level filtering, complete project lifecycle management from posting to delivery, an interactive Kanban-style workspace for task tracking, and integrated real-time messaging between project stakeholders. Built with a clean separation of concerns, the application employs a three-tier architecture consisting of a vanilla HTML/CSS/JavaScript frontend, a PHP REST API backend, and a Microsoft SQL Server database.

The platform supports two distinct user roles with tailored experiences. Clients can post projects with specified budget tiers and required developer levels, browse and hire developers directly, review incoming applications, and manage project workspaces with read-only task visibility. Developers can create detailed profiles showcasing their skills, experience level, portfolio, and social links; apply to projects or respond to client hire requests; and actively manage tasks within accepted project workspaces including adding, moving, and deleting Kanban cards. The dual hiring pathway, where either the client initiates contact or the developer applies proactively, creates a flexible matching system that accommodates different collaboration preferences.

From a technical standpoint, the project demonstrates proficiency in full-stack web development without relying on heavy frameworks. The frontend leverages modern CSS features including custom properties, CSS-only filtering systems, glassmorphism effects, and responsive grid layouts, all orchestrated by a single JavaScript file that manages application state through localStorage and communicates with the backend via the Fetch API. The PHP API layer implements transactional data operations with parameterized queries, bcrypt password hashing, and session-based authentication. The SQL Server database enforces data integrity through CHECK constraints, cascading deletes, and a well-normalized schema with an ISA (inheritance) pattern for the user role hierarchy.

2. Technology Stack

ProjectEngine employs a carefully selected technology stack that balances development simplicity with functional capability. The choice to avoid heavy frontend frameworks and ORMs was intentional, resulting in a lightweight, portable codebase that can be deployed on any standard XAMPP installation. Each layer of the stack was chosen for its reliability, community support, and appropriateness for the application scale.

Layer	Technology	Version / Details	Purpose
Frontend Markup	HTML5	Semantic HTML	8 page templates with form validation

Frontend Styling	CSS3	Custom Properties, Grid, Flexbox	Responsive design, glassmorphism, animations
Frontend Logic	Vanilla JavaScript	ES6+, Fetch API	State management, API communication, DOM manipulation
Backend API	PHP	8.x with sqlsrv extension	20 RESTful endpoints, session authentication
Database	Microsoft SQL Server	2019+ Express	9 tables with constraints, indexes, and seed data
Web Server	Apache (XAMPP)	Cross-platform	Local development and deployment environment
Database Driver	ODBC Driver 17	PHP sqlsrv extension	Parameterized queries, transaction support

Table 1: Technology Stack Overview

3. Architecture and System Design

3.1 Three-Tier Architecture

ProjectEngine follows a classic three-tier architecture pattern that cleanly separates presentation, business logic, and data persistence into distinct layers. The frontend tier consists of static HTML pages enhanced by a shared CSS stylesheet and a single JavaScript file that handles all client-side interactivity. The application tier is a collection of PHP scripts that expose a RESTful JSON API, with each endpoint responsible for a specific business operation. The data tier is a SQL Server database that stores all persistent application state. Communication between the frontend and API tiers uses standard HTTP requests with JSON payloads, while the API tier communicates with the database using parameterized SQL queries through the PHP sqlsrv extension.

Tier	Components	State Management	Communication
Presentation (Browser)	8 HTML pages, style.css, script.js	localStorage (isLoggedIn, userRole, currentUser)	fetch() / FormData POST, JSON responses
Application (PHP API)	20 endpoints + db_connect.php	$_SESSION (user_id, email, role, full_name)	sqlsrv_query() with parameterized SQL
Data (SQL Server)	ProjectEngineDB, 9 tables	Persistent storage with ACID compliance	SQL queries with CHECK constraints and cascading deletes

Table 2: Three-Tier Architecture Breakdown

3.2 Authentication and Authorization

The platform implements a dual-layer authentication system. On the server side, PHP sessions manage user identity through the $_SESSION superglobal, which stores the user ID, email address, assigned role, full name, and a role-specific identifier (client_id or dev_id). Every protected API endpoint begins by checking for the presence of $_SESSION["user_id"] and redirects unauthenticated requests accordingly. Password security is handled through PHP's native PASSWORD_BCRYPT algorithm during registration, with password_verify() used for authentication during login. The registration process wraps user creation and role-specific profile insertion in a database transaction, ensuring atomicity even when linking developer skills.

On the client side, the application mirrors session state in the browser's localStorage, storing three keys: isLoggedIn (a boolean flag), userRole (either "Client" or "Developer"), and currentUser (a JSON object with user details). The script.js file implements a session protection mechanism that runs on every page load, checking localStorage values and redirecting unauthenticated users away from protected pages like the dashboard, workspace, and settings. While this client-side check provides a smooth user experience, it is important to note that it operates independently of the server session. A scenario where the PHP session expires but localStorage persists would result in the UI displaying an authenticated state while API calls return authentication errors. A more robust approach would involve synchronizing client state with periodic session validation calls to the server.

Authorization is primarily role-based, enforced at the API level. Endpoints such as post_project.php explicitly verify that the session user has the "Client" role before allowing project creation. Similarly, hire_developer.php requires a Client session, while respond_hire.php requires a Developer session. Task deletion (delete_task.php) is restricted to developers only, reflecting the business rule that developers manage workspace tasks while clients have read-only access to the task board.

3.3 Data Flow Patterns

All frontend-to-backend communication follows a consistent pattern using the Fetch API. For read operations, the frontend sends GET requests with query parameters and receives JSON responses containing a "success" boolean flag and the requested data. For write operations, the frontend sends POST requests with FormData payloads and receives JSON responses confirming the operation result. Error handling follows HTTP status code conventions: 405 for incorrect HTTP methods, 401 for authentication failures, 404 for missing resources, 409 for conflicts such as duplicate applications, 422 for validation errors, and 500 for server-side failures. The PHP endpoints suppress HTML error output and consistently return JSON, even for error conditions, ensuring that the frontend can always parse the response predictably.

State synchronization between the client and server occurs at key transition points. Upon successful registration or login, the server sets $_SESSION variables and the client stores parallel data in

localStorage. When the user navigates between pages, the client-side session protection reads from localStorage to determine access rights and personalize the UI. The logout endpoint destroys the PHP session, and the corresponding client-side handler clears localStorage before redirecting to the landing page. For workspace chat, the client polls the server every five seconds via setInterval, creating a near-real-time messaging experience without requiring WebSocket infrastructure.

4. Database Design

4.1 Schema Overview

The ProjectEngineDB database consists of nine tables organized around a central Users supertype with Clients and Developers as role-specific subtypes, following the ISA (Inheritance) pattern. This design allows shared authentication fields to reside in the Users table while role-specific attributes are segregated into their respective tables. The Clients and Developers tables share their primary key with the Users table (user_id), establishing a one-to-one relationship enforced by the ON DELETE CASCADE constraint, which ensures that deleting a user automatically removes their role-specific profile. The schema is fully normalized to third normal form, with the many-to-many relationship between Developers and Skills resolved through the Developer_Skills junction table.

Data integrity is enforced through a comprehensive set of CHECK constraints. The Users table restricts the role column to "Client" or "Developer". The Developers table constrains the level column to one of four values: Trainee, Junior, Mid, or Senior. Project status values are limited to Pending, Active, and Completed, while application statuses cycle through Applied, Accepted, and Rejected. Task statuses follow the Kanban pattern with To Do, In Progress, and Done. These CHECK constraints prevent invalid data from entering the database regardless of whether the API validation catches it, providing a defense-in-depth approach to data quality.

4.2 Entity Relationship Summary

Parent Entity	Child Entity	Cardinality	Delete Rule	Description
Users	Clients	1 : 0..1	CASCADE	ISA pattern: client profile
Users	Developers	1 : 0..1	CASCADE	ISA pattern: developer profile
Developers	Developer_Skills	1 : N	CASCADE	Developer skill associations
Skills	Developer_Skills	1 : N	CASCADE	Skill references
Clients	Projects	1 : N	CASCADE	Client-owned projects

Projects	Project_Applications	1 : N	CASCADE	Project applications
Developers	Project_Applications	1 : N	NO ACTION	Prevents cascade loops
Projects	Workspace_Tasks	1 : N	CASCADE	Kanban tasks
Projects	Workspace_Messages	1 : N	CASCADE	Chat messages
Users	Workspace_Messages	1 : N	NO ACTION	Message sender reference

Table 3: Entity Relationships and Delete Rules

4.3 Table Definitions

4.3.1 Users

The Users table serves as the central authentication and identity store, implementing the supertype in the ISA pattern. Every registered account creates exactly one row in this table, with the role column determining which subtype table (Clients or Developers) holds the extended profile. The email column has a UNIQUE constraint to prevent duplicate registrations, and the password_hash column stores bcrypt-hashed passwords, never plaintext. The auto-incrementing user_id serves as the primary key and is referenced by all other tables that need to identify the user, including the role-specific subtype tables and the Workspace_Messages table for chat sender identification.

Column	Type	Constraints	Description
user_id	INT IDENTITY	PRIMARY KEY	Auto-generated unique identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address, used for login
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password
role	VARCHAR(20)	NOT NULL, CHECK IN (Client, Developer)	User role determining access permissions
created_at	DATETIME	DEFAULT GETDATE()	Account creation timestamp

Table 4: Users Table Schema

4.3.2 Developers

The Developers table extends the Users supertype with professional profile attributes. The primary key (dev_id) is also a foreign key referencing Users.user_id, creating a true one-to-one ISA relationship. The

level column uses a CHECK constraint to restrict values to Trainee, Junior, Mid, or Senior, which directly drives the platform's filtering system and visual badge rendering on the frontend. The is_booked column is a critical business logic flag that prevents a developer from being hired for multiple projects simultaneously. When a hire is accepted (through either review_application.php or respond_hire.php), this flag is set to 1, and the hire_developer.php endpoint checks this flag before creating a new application. The hourly_rate, portfolio_url, github_url, and linkedin_url fields provide comprehensive professional information displayed on the developer profile page.

Column	Type	Constraints	Description
dev_id	INT	PK, FK to Users, ON DELETE CASCADE	Same as user_id (ISA pattern)
full_name	VARCHAR(100)	NOT NULL	Display name for the developer
level	VARCHAR(20)	CHECK IN (Trainee, Junior, Mid, Senior)	Experience level tier
hourly_rate	DECIMAL(10,2)	NULL	Hourly billing rate in USD
portfolio_url	VARCHAR(500)	NULL	Portfolio website URL
job_title	VARCHAR(100)	NULL	Professional job title
github_url	VARCHAR(500)	NULL	GitHub profile URL
linkedin_url	VARCHAR(500)	NULL	LinkedIn profile URL
bio	NVARCHAR(MAX)	NULL	Professional biography text
is_booked	BIT	DEFAULT 0	Availability flag (0=available, 1=booked)

Table 5: Developers Table Schema

4.3.3 Clients

The Clients table mirrors the Developers table in its ISA relationship with Users, but contains significantly fewer attributes. This reflects the business model where clients primarily need to identify themselves and their organization, while the detailed professional profiling is reserved for developers. The company_name field is the most important attribute, as it is displayed on project listings and workspace headers. The contact_number provides an optional communication channel outside the platform. Like the Developers table, the primary key client_id doubles as a foreign key to Users.user_id with ON DELETE CASCADE, ensuring that account deletion cleanly removes the client profile without orphaned records.

4.3.4 Projects

The Projects table is the central entity for the platform's core business logic. Each project is owned by a client (via the client_id foreign key), has a lifecycle status that transitions through Pending, Active, and Completed states, and specifies requirements including budget_tier and required_level. The status transitions are managed by the API: a newly posted project starts as Pending, becomes Active when a developer is hired (through either review_application.php or respond_hire.php), and can be marked Completed through the workspace. The budget_tier and required_level fields act as matching criteria that help developers find suitable projects and help clients specify their expectations. The CASCADE delete from Clients ensures that deleting a client removes all their projects and associated data.

Column	Type	Constraints	Description
project_id	INT IDENTITY	PRIMARY KEY	Auto-generated project identifier
client_id	INT	FK to Clients, ON DELETE CASCADE	Owning client reference
title	VARCHAR(300)	NOT NULL	Project title (max 300 chars)
description	NVARCHAR(MAX)	NULL	Detailed project description
budget_tier	VARCHAR(50)	NULL	Budget classification
required_level	VARCHAR(20)	CHECK IN (Trainee, Junior, Mid, Senior)	Minimum developer level required
status	VARCHAR(20)	CHECK IN (Pending, Active, Completed)	Project lifecycle status

Table 6: Projects Table Schema

4.3.5 Supporting Tables

The remaining five tables provide the supporting data structures. The Skills table stores a canonical list of technology skills (seeded with HTML/CSS, JavaScript, React, Vue.js, Node.js, Python, Flutter, React Native, Kotlin, and SQL Server), with a UNIQUE constraint on skill_name to prevent duplicates. The Developer_Skills junction table implements the many-to-many relationship between Developers and Skills using a composite primary key (dev_id, skill_id) with CASCADE deletes from both parent tables. The registration API uses an upsert pattern for skills: when a developer lists skills during registration or profile update, any new skill names are inserted into the Skills table if they do not already exist, then linked to the developer through Developer_Skills.

The Project_Applications table tracks the hiring workflow with a UNIQUE constraint on (project_id, dev_id) to prevent duplicate applications. Its status column transitions through Applied, Accepted, and Rejected states. When a client accepts an application through review_application.php, a transaction updates the application status to Accepted, activates the project, books the developer, and auto-rejects all other pending applications for the same project. The Workspace_Tasks table implements the Kanban

board with To Do, In Progress, and Done statuses, tracking both created_at and updated_at timestamps. The Workspace_Messages table stores chat messages with a foreign key to the sender's user_id using NO ACTION on delete, preventing accidental data loss when user accounts are deleted while preserving the message history for remaining project participants.

4.4 Indexes

Index Name	Table	Column	Type	Rationale
IX_Projects_ClientID	Projects	client_id	NONCLUSTERED	Speeds up dashboard project listing by client
IX_WkMsg_ProjectID	Workspace_Messages	project_id	NONCLUSTERED	Optimizes chat message retrieval per project

Table 7: Database Indexes

5. API Reference

The ProjectEngine API consists of 20 PHP endpoints, each mapped to a specific business operation. All endpoints follow a consistent design pattern: they validate the HTTP method, check authentication where required, validate input parameters, execute the database operation (often within a transaction), and return a JSON response with a "success" boolean flag. Error responses include a "message" field with a human-readable description and, for validation errors, an "errors" array detailing each validation failure. This section provides a comprehensive reference for every endpoint, organized by functional domain.

5.1 Authentication Endpoints

Endpoint	Method	Auth	Parameters	Returns
register.php	POST	None	full_name, email, password, role + role-specific fields	user_id, role, redirect
login.php	POST	None	email, password	user_id, role, name, redirect
logout.php	POST	None	None	redirect to index.html

Table 8: Authentication Endpoints

The registration endpoint is the most complex in the authentication domain. It accepts different parameter sets depending on the requested role. For Client registrations, the required fields are full_name, email, password, role (set to "Client"), and company_name. For Developer registrations, the

system additionally collects level (Trainee/Junior/Mid/Senior), skills (a comma-separated string), hourly_rate, portfolio_url, job_title, github_url, linkedin_url, and bio. The endpoint validates the email format, enforces a minimum password length of six characters, and verifies that the role matches one of the two allowed values. On success, the entire operation is wrapped in a SQL Server transaction: the user record is inserted first, then the role-specific profile, and for developers, the skills are parsed and upserted into the Skills table with links created in Developer_Skills. If any step fails, the entire transaction is rolled back, ensuring no partial records remain in the database.

5.2 Project Management Endpoints

Endpoint	Method	Auth	Parameters	Returns
post_project.php	POST	Client	title, description, budget_tier, required_level	project_id, status
get_projects.php	GET	Session	None (session-based)	projects[] with role-aware data
apply_project.php	POST	Developer	project_id	application_id, status

Table 9: Project Management Endpoints

The post_project.php endpoint allows clients to create new projects. It validates that the session user has the Client role, checks that the title does not exceed 300 characters, and inserts the project with a default status of "Pending." The endpoint uses the OUTPUT INSERTED.project_id SQL Server clause to return the newly created project identifier within the same transaction. The get_projects.php endpoint is role-aware: for clients, it returns all projects they posted with their current statuses; for developers, it returns projects they have applied to, including the application status. This design allows the dashboard to display different information depending on the user role without requiring separate endpoints.

5.3 Developer and Hiring Endpoints

Endpoint	Method	Auth	Parameters	Returns
get_developers.php	GET	None	None	developers[] with skills
get_profile.php	GET	Optional	id (query param)	developer profile + client projects
hire_developer.php	POST	Client	dev_id, project_id	success message
review_application.php	POST	Client	application_id, status	updated status

respond_hire.php	POST	Developer	project_id, status	success flag

Table 10: Developer and Hiring Endpoints

The hiring workflow involves two distinct pathways that converge on the same Project_Applications table. In the client-initiated path, hire_developer.php creates an application record after verifying the developer is not already booked and the project does not already have a pending or accepted application. In the developer-initiated path, apply_project.php allows a developer to apply to a pending project, also creating an application record. Both pathways lead to acceptance decisions: review_application.php lets clients accept or reject developer applications, while respond_hire.php lets developers accept or reject client hire requests. When an application is accepted through either pathway, a transaction activates the project, marks the developer as booked, and rejects all other pending applications for the same project, ensuring exclusive developer-project pairing.

5.4 Workspace Endpoints

Endpoint	Method	Auth	Parameters	Returns
get_workspace.php	GET	Session	id (query param)	project info + tasks[]
add_task.php	POST	Session	project_id, title, description	success flag
move_task.php	POST	Session	task_id, status	success flag
delete_task.php	POST	Developer	id (task_id)	success message

Table 11: Workspace Endpoints

The get_workspace.php endpoint provides all data needed to render the Kanban board and chat interface. It performs different JOIN operations depending on the user role: for clients, it uses a LEFT JOIN to retrieve the partner developer information and application status, returning early with an app_status of "No_Hire" if no developer has been hired yet. For developers, it uses an INNER JOIN to retrieve only accepted applications, ensuring developers only see workspaces for projects they are actively working on. The task management endpoints follow simple CRUD patterns: add_task.php creates a new task with default "To Do" status, move_task.php transitions a task between columns by updating the status and updated_at timestamp, and delete_task.php removes a task entirely (restricted to developers only).

5.5 Messaging Endpoints

Endpoint	Method	Auth	Parameters	Returns

get_messages.php	GET	Session	id (project_id, query)	messages[] with sender info
send_message.php	POST	Session	project_id, message	success flag

Table 12: Messaging Endpoints

The messaging system is implemented as a project-scoped chat, where all messages are tied to a specific project_id rather than being direct user-to-user conversations. The get_messages.php endpoint JOINs the Workspace_Messages table with Users, Developers, and Clients to resolve each sender's display name based on their role, and adds an is_mine boolean flag based on the current session user_id, enabling the frontend to distinguish between sent and received messages for visual differentiation. The frontend polls this endpoint every five seconds using setInterval, creating a near-real-time chat experience. The sent_at timestamp is converted to VARCHAR format for JSON compatibility, as SQL Server DATETIME objects cannot be directly encoded in JSON by the PHP sqlsrv driver.

5.6 Profile and Account Endpoints

Endpoint	Method	Auth	Parameters	Returns
get_my_profile.php	GET	Session	None (session-based)	profile data for settings page
update_profile.php	POST	Session	Role-specific fields + skills	success flag
delete_account.php	POST	Session	password (verification)	success + redirect

Table 13: Profile and Account Endpoints

The update_profile.php endpoint handles role-specific updates within a transaction. For developers, it updates the Developers table fields, then deletes all existing Developer_Skills records and re-inserts them from the comma-separated skills parameter, using the same upsert pattern as registration to create new Skills entries as needed. The delete_account.php endpoint implements a safe deletion workflow: it first verifies the user's password with password_verify(), then checks business rules (clients with active projects or developers with accepted applications cannot delete their accounts), and finally executes a transactional deletion that manually removes records from Developer_Skills, Project_Applications, Developers/Clients, and Workspace_Messages before removing the Users record. The session is then destroyed and the client is redirected to the landing page.

6. Frontend Documentation

6.1 Page Inventory

The frontend consists of eight HTML pages, each serving a specific user-facing function. All pages share the same CSS stylesheet (style.css) and JavaScript file (script.js), which together form a single-page-like experience without using a JavaScript framework. Navigation between pages is handled through standard HTML links with JavaScript guards for protected routes. Each page includes the same navigation bar with role-aware links, and the script.js file dynamically shows or hides elements based on the user's authentication state and role.

Page	File	Auth Required	Key Features
Landing	index.html	No	Hero section, how-it-works flow, developer level cards
Login	login.html	No	Email/password form, session creation
Register	register.html	No	Role toggle (Client/Developer), role-specific fields
Dashboard	dashboard.html	Yes	Project list, status badges, hire request actions
Developers	developers.html	No	Developer grid, CSS-only level filtering, profile links
Profile	profile.html	Optional	Developer detail view, GitHub API sync, hire action
Workspace	workspace.html	Yes	Kanban board, task CRUD, chat widget, progress bar
Post Project	post-project.html	Client	Project creation form with level and budget selection
Settings	settings.html	Yes	Profile editing, account deletion with password verification

Table 14: Frontend Page Inventory

6.2 CSS Design System

The stylesheet implements a comprehensive design system using CSS custom properties (variables) for consistent theming across all pages. The primary color palette uses navy (#001F54) and gold (#E9B851), with a light gray background (#F5F7FA) and white card surfaces. The Outfit font family from Google Fonts is used exclusively, with weights ranging from 300 (Light) to 800 (ExtraBold), providing clear typographic hierarchy without mixing multiple font families. All interactive elements follow a pill-shaped design language with 50px border-radius, creating a cohesive and modern visual identity.

Notable CSS patterns include a CSS-only developer filtering system on the developers page, which uses hidden radio inputs and the :checked sibling selector to show or hide developer cards by level without any JavaScript. The registration page employs a similar CSS-only role toggle for showing and hiding Client/Developer-specific form fields, though the JavaScript handler also manages this for robustness. The Kanban board uses flexbox columns with subtle hover effects and a pulsing indicator on the "In Progress" column. The chat widget is implemented as a fixed-position panel with CSS transform animations for smooth open/close transitions. The progress bar uses a gradient fill with a shimmer animation to indicate active work. Level badges have four distinct color schemes: green for Trainee, navy for Junior, dark navy for Mid, and gold with shadow for Senior.

6.3 JavaScript Architecture

The entire frontend application logic is contained in a single script.js file (approximately 1,048 lines) that runs inside a DOMContentLoaded event listener. This monolithic approach, while not typical of production applications, provides simplicity and avoids module loading complexity for a project of this scale. Global functions for onclick handlers (addTask, moveTask, deleteTask, respondHire) are attached to the window object to allow inline HTML event binding. The script manages application state through three localStorage keys (isLoggedIn, userRole, currentUser) and communicates with the API exclusively through the Fetch API.

Key JavaScript features include a navbar auto-hide mechanism that conceals the navigation bar when the user scrolls down more than 100 pixels and reveals it on upward scroll, creating more screen space for content. The session protection system runs on every page load, checking localStorage and redirecting unauthenticated users away from protected pages while populating role-specific UI elements like avatar initials and sidebar names. The dashboard rendering logic dynamically generates project cards with status-dependent badges and action buttons based on the user role. The workspace rendering system handles four distinct client states (No_Hire, Applied/Pending, Rejected, and Accepted) and provides full task management for accepted developers. The notification system uses a toast-style pattern with slide-in animations for success and error feedback.

7. User Flows

7.1 Client Journey

The client journey begins with registration on the platform, where the user selects the Client role and provides their full name, email, password, and company name. After successful registration, the client is redirected to the dashboard, which initially displays an empty state with a call-to-action to post their first project. The client navigates to the Post Project page, fills in the project title, description, required developer level, and budget tier, and submits the form. The new project appears on the dashboard with a "Looking for Devs" status badge, indicating it is pending and awaiting developer interest.

The client can browse the Developers page to discover available talent. The CSS-only filtering system allows them to narrow results by experience level. Clicking on a developer card opens the full profile page, which displays the developer's level, skills, hourly rate, portfolio, GitHub, and LinkedIn links. If the client has pending projects, a project selector dropdown and "Hire Developer" button appear on the profile page, allowing the client to send a hire request linked to a specific project. Back on the dashboard, when a developer applies to the client's project, the client sees the application and can accept or reject it. Accepting an application triggers a transaction that activates the project, books the developer, and rejects all other pending applications.

Once a project has an accepted developer, the client can enter the workspace, where they see the Kanban board displaying tasks organized into To Do, In Progress, and Done columns. Clients have read-only access to the task board but can use the chat widget to communicate with the developer in real time. The sprint progress bar at the top of the workspace provides a weighted progress indicator: Done tasks contribute 100%, In Progress tasks contribute 50%, and To Do tasks contribute 0%. From the Settings page, clients can update their company name and contact number, or permanently delete their account after password verification (blocked if they have active projects).

7.2 Developer Journey

The developer journey starts with a more detailed registration process that captures professional information including experience level (Trainee, Junior, Mid, or Senior), a comma-separated list of skills, hourly rate, portfolio URL, job title, GitHub URL, LinkedIn URL, and a professional biography. After registration, the developer is redirected to the dashboard, which shows any hire requests received from clients. Each hire request displays the project title and provides Accept and Deny buttons. Accepting a hire request triggers the same activation transaction as the client-side acceptance flow.

In the workspace, developers have full task management capabilities. They can add new tasks through a prompt dialog, move tasks between Kanban columns using directional buttons, and delete tasks entirely. The chat widget provides real-time communication with the client. The developer's profile page features a GitHub API integration: if the developer has not provided a bio but has linked their GitHub profile, the system automatically fetches their GitHub biography from the public API endpoint. The Settings page allows developers to update all profile fields, including skills (which uses the same upsert pattern as registration). Account deletion is available with the same safety checks as the client flow, preventing deletion if the developer has accepted applications.

7.3 Dual Hiring Pathways

A distinctive feature of ProjectEngine is its support for two symmetric hiring pathways. In the client-initiated pathway, the client discovers a developer through the Developers page, views their profile, selects a project from their pending projects, and clicks "Hire Developer." This creates a Project_Applications record with status "Applied," which appears on the developer's dashboard as a hire request. The developer then responds by accepting or rejecting through the respond_hire.php endpoint. In the developer-initiated pathway, the developer applies to a pending project through the

apply_project.php endpoint, and the client reviews and responds through the review_application.php endpoint. Both pathways converge on the same database table and trigger the same activation transaction on acceptance, providing flexibility in how collaborations are initiated while maintaining data consistency.

8. Security Considerations

8.1 Implemented Security Measures

The platform implements several important security measures. Password storage uses PHP's native PASSWORD_BCRYPT algorithm, which incorporates automatic salt generation and a cost factor that makes brute-force attacks computationally expensive. All SQL queries use parameterized statements through the PHP sqlsrv extension, effectively preventing SQL injection attacks by separating query logic from data. The database connection configuration suppresses HTML error output and logs errors to a dedicated file (api/php_error.log), preventing sensitive server information from being exposed in API responses. Input validation at the API layer enforces email format, password length, and enum value restrictions before data reaches the database.

8.2 Identified Security Gaps

Issue	Severity	Impact	Recommended Mitigation
No CSRF Protection	High	Cross-site request forgery attacks could perform unauthorized actions on behalf of authenticated users	Implement anti-CSRF tokens in all POST forms and validate them server-side
No Rate Limiting	High	Brute-force attacks against the login endpoint are unrestricted	Add rate limiting middleware with exponential backoff and account lockout
XSS via innerHTML	High	Malicious project titles or messages could inject and execute arbitrary JavaScript	Replace innerHTML with textContent or implement output encoding/sanitization
Session/localStorage Mismatch	Medium	Expired PHP sessions with persistent localStorage create confusing UI states and potential auth bypass	Implement periodic session validation calls and sync client state with server
No HTTPS Enforcement	Medium	Credentials and session tokens transmitted in plaintext over HTTP	Enforce HTTPS through server configuration and HSTS headers
Plaintext DB Credentials	Low	Database credentials hardcoded in db_connect.php are visible in source code	Use environment variables or a secrets manager for credential storage

Table 15: Security Gap Assessment

9. Known Issues and Limitations

9.1 Functional Issues

Several functional issues have been identified during analysis of the codebase. The most significant is the absence of a frontend interface for the developer-to-project application pathway. While the apply_project.php endpoint exists and is fully functional, there is no button or link on any HTML page that allows developers to browse and apply to pending projects. The only hiring mechanism exposed in the UI is the client-initiated "Hire Developer" button on the profile page. This means the dual hiring pathway is only half-implemented from a user interface perspective, despite being fully supported on the backend.

Another functional issue involves duplicate event handlers on the registration form. The script.js file contains both a generic form handler (lines 96-164) that intercepts all form submissions and a specific registerForm handler (lines 187-206) that also binds to the registration form. This duplication means the registration API may be called twice for a single form submission, potentially creating duplicate application errors or confusing user feedback. Additionally, the "Category" select element on the post-project.html page has no name attribute, so the selected category is never sent to the API or stored in the database, rendering the field purely decorative.

9.2 Architectural Limitations

The monolithic script.js architecture, while simple, creates maintenance challenges as the application grows. The single 1,048-line file contains all page-specific logic, making it difficult to isolate and test individual features. The lack of pagination in the get_developers.php and get_projects.php endpoints means that as the platform scales and the number of records grows, these endpoints will return increasingly large JSON payloads, degrading performance for both the server and the client. The chat system relies on polling (every five seconds) rather than a push-based mechanism like WebSockets, which creates unnecessary server load and introduces message delivery latency.

The database design does not currently support project completion workflows. While the Projects table includes a "Completed" status value, there is no API endpoint or UI mechanism for transitioning a project from "Active" to "Completed." This means that once a project is activated by hiring a developer, it remains in that status indefinitely. Similarly, the developer is_booked flag is set to 1 when hired but is never reset, meaning a developer who completes a project would remain permanently booked and unable to be hired for new projects. These are critical limitations for a platform intended to support the full project lifecycle.

10. Setup and Installation Guide

10.1 Prerequisites

Before installing ProjectEngine, ensure your system meets the following prerequisites. You will need XAMPP with Apache and PHP 8.x installed, including the PHP sqlsrv extension for SQL Server connectivity. Microsoft SQL Server 2019 Express (or later) must be installed and running, with the SQLEXPRESS01 instance name configured as referenced in the database connection file. The ODBC Driver 17 for SQL Server must be installed on the system for the PHP sqlsrv extension to communicate with the database. A modern web browser (Chrome, Firefox, Edge, or Safari) is required for the frontend, which uses ES6+ features including the Fetch API, template literals, and arrow functions.

10.2 Installation Steps

1. **Step 1:** Extract the project files into the XAMPP htdocs directory (e.g., C:\xampp\htdocs\ProjectEngine).
2. **Step 2:** Open SQL Server Management Studio and connect to the .\SQLEXPRESS01 instance.
3. **Step 3:** Execute the database/schema.sql script to create the ProjectEngineDB database, login user, and all tables with seed data.
4. **Step 4:** Verify the database connection by navigating to http://localhost/ProjectEngine/api/get_developers.php in your browser. A JSON response with an empty or populated developers array confirms successful connectivity.
5. **Step 5:** Start the Apache server through the XAMPP Control Panel.
6. **Step 6:** Access the application by navigating to http://localhost/ProjectEngine/index.html in your web browser.
7. **Step 7:** Register a new account (either Client or Developer) to begin using the platform.

10.3 Configuration

The primary configuration file is api/db_connect.php, which contains the database connection parameters. The default configuration assumes SQL Server is running on the local machine with the instance name .\SQLEXPRESS01, using SQL Server Authentication with the username "ProjectEngineUser" and password "Engine@2026!". If your SQL Server instance uses a different name, you must update the $serverName variable in db_connect.php accordingly. Similarly, if you change the database name, login credentials, or server address, all modifications should be made in this single file, as every API endpoint includes it for database connectivity. The connection also sets WarningsReturnAsErrors to 0 to handle ODBC Driver 17 IM006 warnings that occur on some configurations, and disables encryption with TrustServerCertificate enabled for local development convenience.

11. File Dependency Map

Understanding the dependencies between files is essential for maintaining and extending the project. The frontend HTML pages all depend on the shared style.css and script.js files. The script.js file, in turn, makes fetch() calls to 18 of the 20 PHP API endpoints (login.php and register.php are called directly from form handlers). All PHP endpoints depend on db_connect.php for database connectivity, which connects to the SQL Server instance hosting the ProjectEngineDB database. The following table provides a complete mapping of file dependencies.

Source File	Depends On
All HTML pages	style.css, script.js
script.js	18 API endpoints (all except db_connect.php, logout.php)
All PHP API files	api/db_connect.php
db_connect.php	SQL Server instance (ProjectEngineDB)
database/schema.sql	SQL Server instance (creates ProjectEngineDB)

Table 16: File Dependency Map